Report on

# Mental Health Domain Classification


**Submitted by:**

**(TEAM 1)**

Geetha A – 160123748001

Siri Nandini A - 160123748002

Pranathi B – 160123748003

**Submitted to**

Mr. Panigrahi Srikanth

Assistant Professor @ Dept of AIML

Chaitanya Bharathi Institute of Technology

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)(Affiliated to Osmania University; Accredited by NBA and NAAC, ISO 9001:2015 Certified Institution), GANDIPET, HYDERABAD 500 075

Website: www.cbit.ac.in


Date of Submission: **22-4-2026**

## ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our faculty mentor, **Mr. Panigrahi Srikanth**, Assistant Professor in the Department of Artificial Intelligence and Machine Learning, for his invaluable guidance, continuous support, and encouragement throughout the development of our project, “Mental Health Domain Classification.” His insightful suggestions and expert supervision played a vital role in the successful completion of this work. We would like to acknowledge the efforts and cooperation of all our team members, whose dedication, collaboration, and mutual support made this project possible.

Geetha A - 160123748001

Siri Nandini A - 160123748002

Pranathi B - 160123748003

# 1. ABSTRACT

Mental health is a critical aspect of overall well-being, yet identifying and categorizing mental health-related concerns from textual data remains a challenging task. With the increasing availability of digital conversations and queries, automated systems can play a significant role in understanding and classifying mental health domains effectively.

This project, titled *"Mental Health Domain Prediction,"* focuses on developing a machine learning-based system to classify mental health-related queries into appropriate domains. The model is trained using the BharatGen MHQA dataset, which consists of diverse mental health-related questions and annotations across multiple categories like - Depression, Trauma, Anxiety & OCD. The dataset enables the system to learn patterns and contextual meanings associated with different mental health issues.

The results demonstrate that the proposed system can effectively classify mental health queries into relevant domains, aiding in better organization and potential support systems. This project highlights the potential of AI-driven solutions in mental health analysis and provides a foundation for building intelligent support tools in the future.

Overall, the system contributes toward improving accessibility and scalability in mental health understanding through automated domain classification.

# TABLE OF CONTENTS


| S.No | Topic | page.no |
|---|---|---|
| 1 | Introduction | 4 |
| 2 | Dataset Description | 5-8 |
| 3 | Methodology | 9-13 |
| 4 | Implementation | 14-19 |
| 5 | Results and Analysis | 20-23 |
| 6 | Conclusion | 24 |
| 7 | Future Work | 24 |
| 8 | Appendix | 25 |
| 9 | References | 26 |

# 2.INTRODUCTION

## 2.1 Background of Mental Health Analysis

Mental health analysis has traditionally relied on manual evaluation by professionals, which can be time-intensive and subjective. With advancements in AI and NLP, it is now possible to analyze large volumes of textual data to identify emotional tone, intent, and domain-specific context.

Recent developments in transformer-based models, such as BERT, have significantly improved the ability of machines to understand human language by capturing contextual and semantic relationships. These models are particularly effective in tasks like text classification, sentiment analysis, and domain prediction. Leveraging such technologies in the mental health domain can assist in early identification, categorization, and support for individuals.

---

## 2.2 Problem Statement

The increasing volume of mental health-related textual data across digital platforms makes it difficult to manually categorize and analyze information efficiently. Existing methods are often limited by scalability issues and lack the ability to capture contextual nuances in human language.

The problem addressed in this project is to develop an automated system that can accurately classify mental health-related queries into predefined domains using NLP techniques. The system should be capable of understanding the context and semantics of the input text to ensure reliable and efficient classification.

---

## 2.3 Objectives of the Project

The main objectives of this project are:

- To develop a machine learning-based system for mental health domain classification.
- To utilize the BharatGen MHQA dataset for training and evaluation.
- To preprocess and clean textual data for effective model performance.
- To implement NLP techniques and transformer-based models for accurate classification.
- To evaluate the model using standard performance metrics such as accuracy, precision, recall, and F1-score.
- To demonstrate the effectiveness of AI in analyzing and organizing mental health-related data.

# 3. DATASET DESCRIPTION

### 3.1 Overview of Datasets (MHQA and MHQA-B)

This project utilizes two datasets from the BharatGen initiative: MHQA and MHQA-B, available at: https://aikosh.indiaai.gov.in/home/datasets/details/bharatgen_mhqa_dataset.html

The **MHQA dataset** is a gold-standard (human-annotated) dataset, consisting of 2,475 high-quality labeled samples. It is used for validation and testing to ensure reliable evaluation of the model.

The **MHQA-B dataset** is a much larger dataset with 56,142 samples and contains pseudo-labeled data, meaning the labels are generated automatically or semi-automatically. This dataset is used entirely for training, allowing the model to learn from a large volume of data.

The classification task involves categorizing mental health queries into four primary domains:

- Anxiety
- Depression
- Obsessive/Compulsive Disorders
- Trauma

---

### 3.2 Dataset Structure and Features

Both datasets follow a structured tabular format with multiple columns representing different attributes of each sample. The key columns include:

- **id**: Unique identifier for each entry
- **topic**: Label representing the mental health domain
- **type**: Type of question
- **question**: Main textual input used for classification
- **option1, option2, option3, option4**: Multiple-choice answer options
- **correct_option**: Correct answer text
- **correct_option_number**: Index of the correct answer

For this project, the **question** field is used as the primary input feature, while the **topic** field serves as the target label for classification.

---

### 3.3 Label Mapping

The categorical labels in the dataset are mapped to numerical values to enable model training:

- Anxiety → 0
- Depression → 1
- Obsessive/Compulsive Disorders → 2
- Trauma → 3

---

### 3.4 Data Splitting Strategy

A structured data splitting approach is followed to ensure proper training and evaluation:

- **Training Set**: Entire MHQA-B dataset (56,142 samples)
- **Validation + Test Set**: MHQA dataset (2,475 samples)

The MHQA dataset is further divided into:

- **Validation Set**: 1,237 samples
- **Test Set**: 1,238 samples

This strategy ensures that the model is trained on a large dataset (pseudo-labeled) while being evaluated on high-quality, human-annotated data for better generalization and reliability.

---

### 3.5 Data Preprocessing

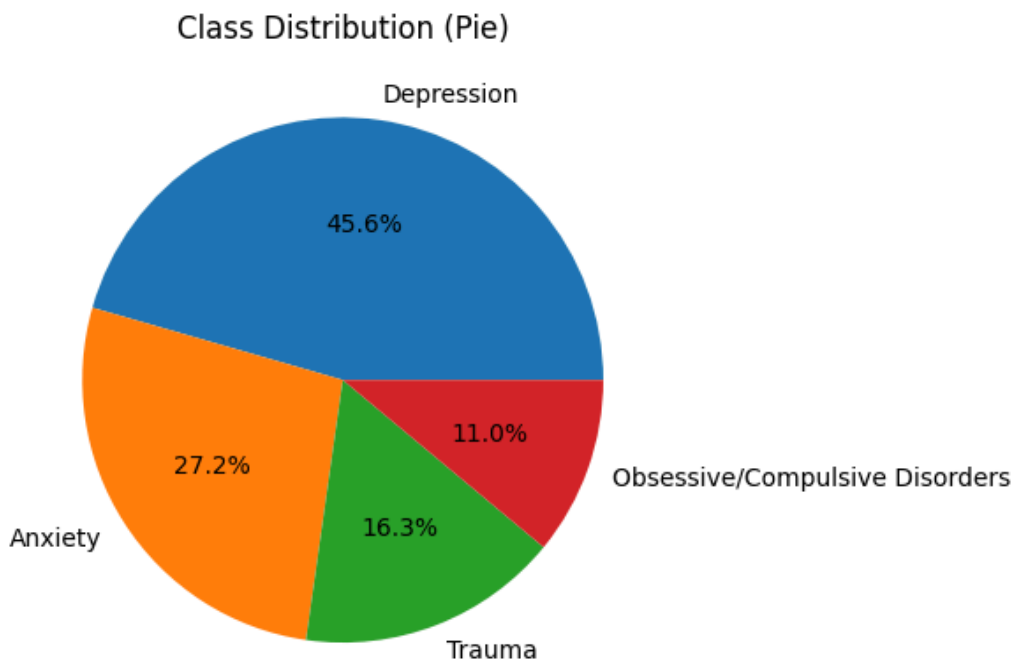
To prepare the dataset for model training, several preprocessing steps were applied:

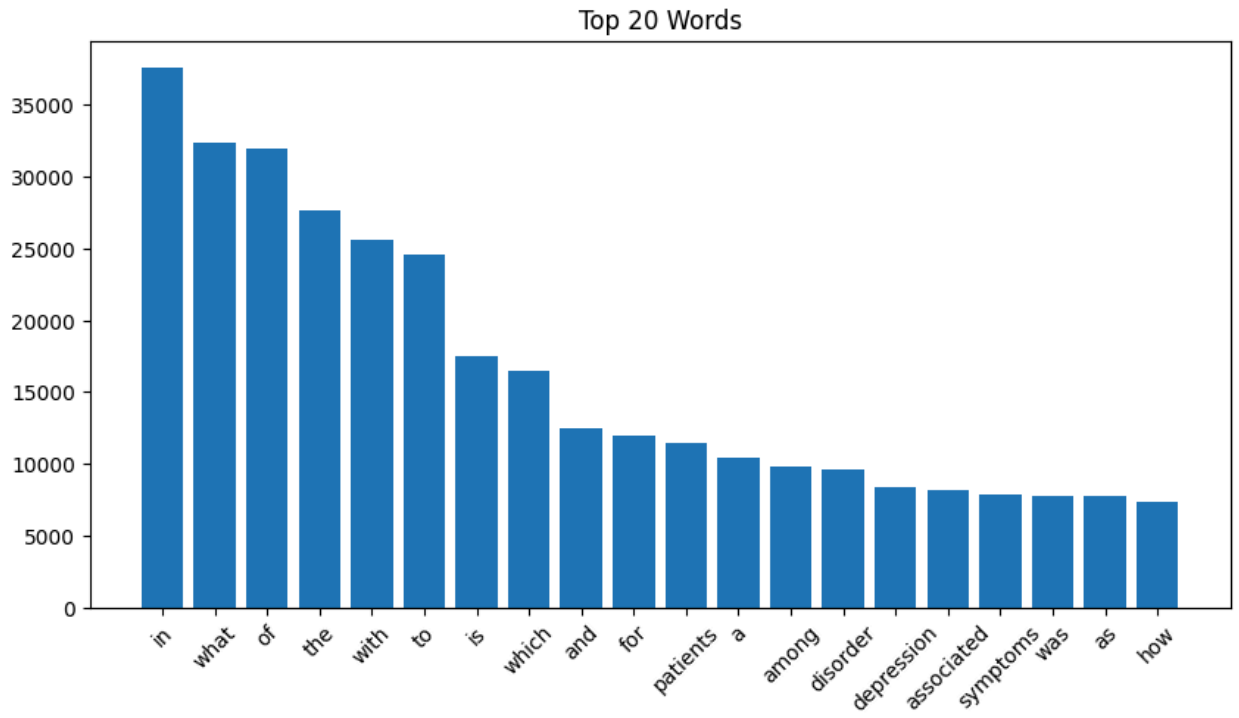
- **Text Cleaning**: Removal of unnecessary characters and noise
- **Lowercasing**: Standardizing all text to lowercase
- **Tokenization**: Splitting text into tokens using a tokenizer
- **Padding and Truncation**: Ensuring uniform input length
- **Encoding**: Converting text into numerical representations using transformer-based tokenizers

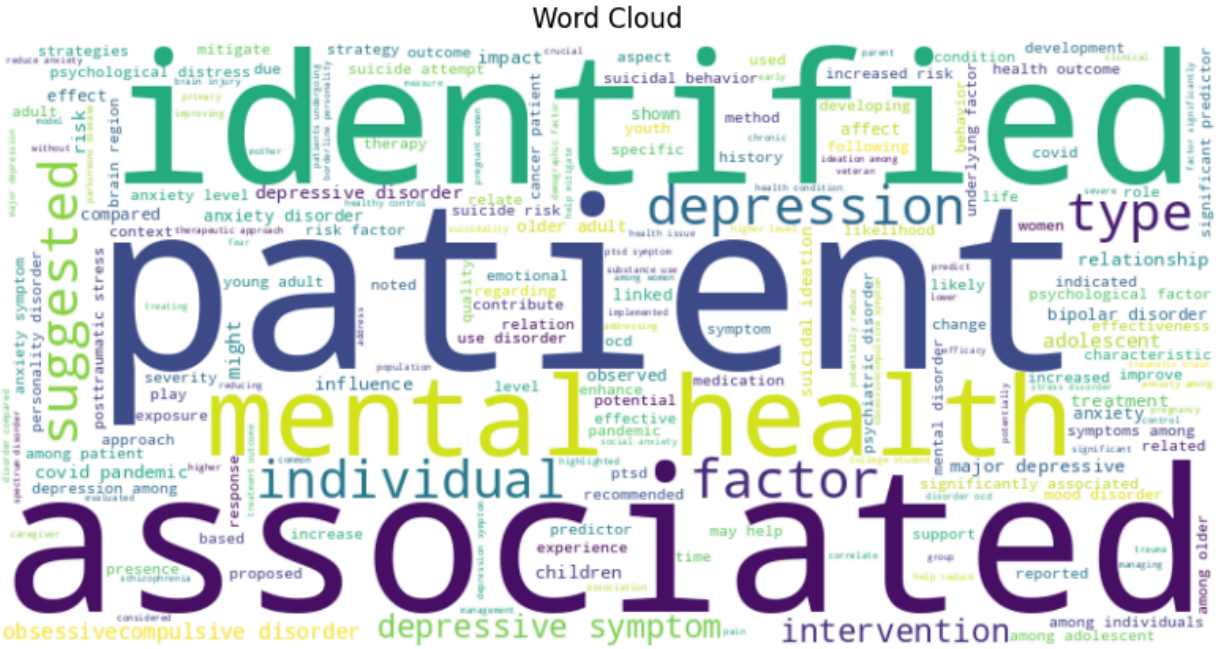
These preprocessing steps improve data quality and ensure compatibility with NLP models such as BERT.

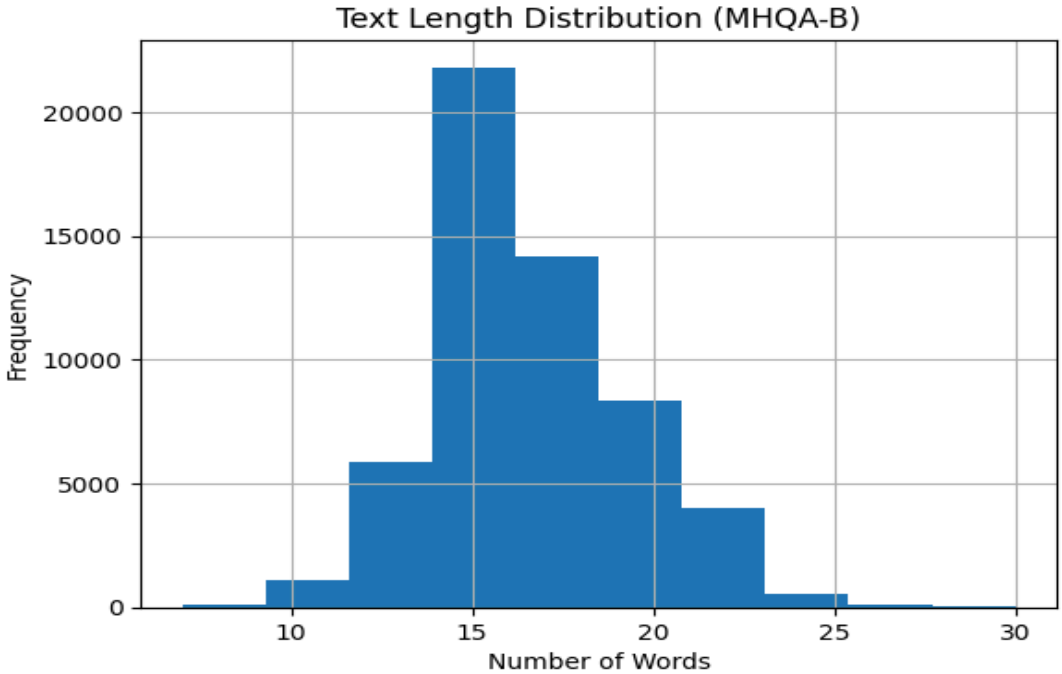
---

### 3.6 Exploratory Data Analysis and Visualization


Class Distribution (Pie)
Depression
45.6%
27.2%
Anxiety
16.3%
Trauma
11.0%
Obsessive/Compulsive Disorders


Top 20 Words
35000
30000
25000
20000
15000
10000
5000
0
in
what
of
the
with
to
is
which
and
for
patients
a
among
disorder
depression
associated
symptoms
was
as
how


Word Cloud
identified
patient
associated
mental health
depression
type
suggested
individual
factor
obsessivecompulsive disorder
depressive symptom
intervention


Text Length Distribution (MHQA-B)
Frequency
Number of Words
0
5000
10000
15000
20000
10
15
20
25
30

# 4. METHODOLOGY

## 4.1 System Overview

The proposed system is designed to classify mental health-related queries into predefined domains using transformer-based Natural Language Processing (NLP) models. The workflow consists of data preprocessing, model training, evaluation, and comparison of multiple transformer architectures.

The system takes a textual query as input and outputs one of the four mental health domains: Anxiety, Depression, Obsessive/Compulsive Disorders, or Trauma.

---

## 4.2 Data Preparation

The dataset is prepared by combining MHQA (gold-labeled) and MHQA-B (pseudo-labeled) datasets. The MHQA-B dataset is used entirely for training, while the MHQA dataset is split equally for validation and testing.

Text preprocessing steps include:

- Conversion to lowercase
- Removal of null values
- Trimming unnecessary spaces

The input feature used is the **question text**, and the target variable is the **topic label**.

---

## 4.3 Label Encoding

Categorical labels are converted into numerical format using a label encoder. To ensure consistency, the encoder is fitted on the combined labels from both MHQA and MHQA-B datasets.

This ensures that all labels are mapped correctly across training, validation, and testing datasets.

---

## 4.4 Model Selection

Multiple transformer-based models were evaluated for performance comparison:

- BERT (bert-base-uncased)
- ELECTRA

- DistilBERT
- RoBERTa

These models are pre-trained language models capable of capturing contextual and semantic information from text.

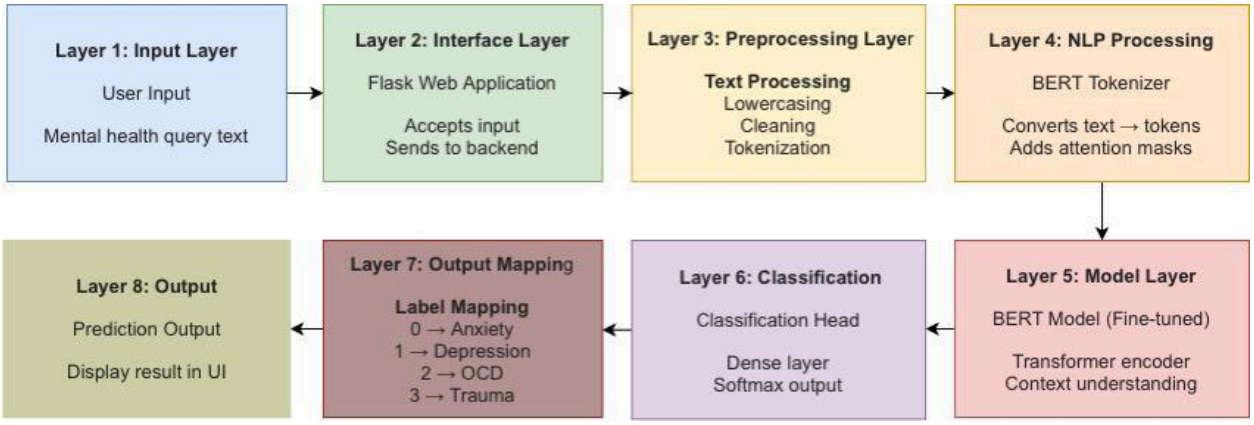
---

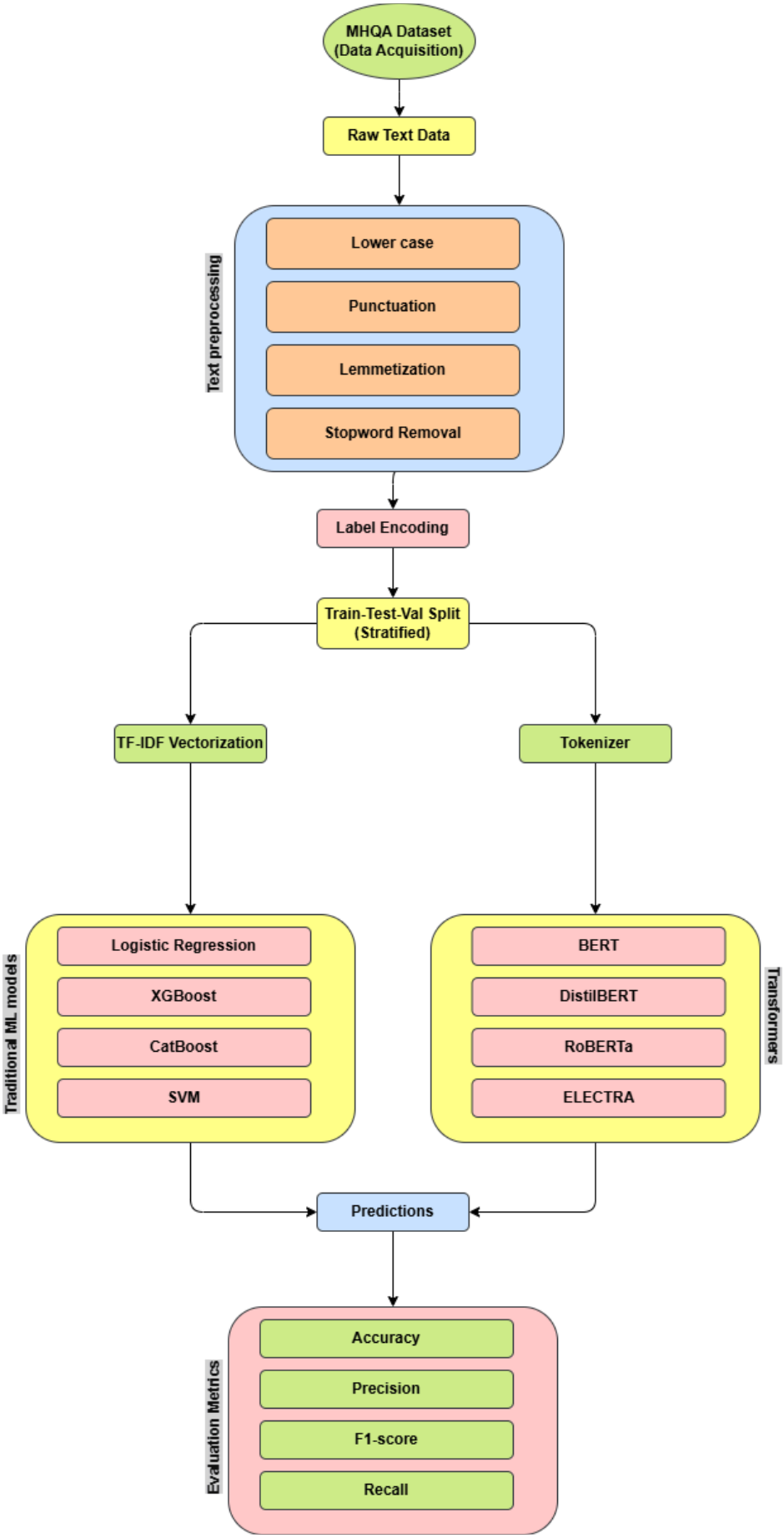
### 4.5 Model Architecture

The selected model, **BERT (bert-base-uncased)**, is used for sequence classification. It consists of:

- Tokenizer for converting text into input tokens
- Transformer encoder layers for contextual understanding
- Classification head for predicting domain labels

The model is fine-tuned on the training dataset for the classification task.




MHQA Dataset
(Data Acquisition)
Raw Text Data
Text preprocessing
Lower case
Punctuation
Lemmetization
Stopword Removal
Label Encoding
Train-Test-Val Split
(Stratified)
TF-IDF Vectorization
Tokenizer
Traditional ML models
Logistic Regression
XGBoost
CatBoost
SVM
Transformers
BERT
DistilBERT
RoBERTa
ELECTRA
Predictions
Evaluation Metrics
Accuracy
Precision
F1-score
Recall

---

### 4.6 Training Configuration

The training process is implemented using the PyTorch and Hugging Face Transformers library .

Key training parameters:

- Maximum sequence length: 256
- Batch size: 16
- Number of epochs: 2
- Learning rate: 2e-5
- Optimizer: AdamW
- Scheduler: Linear learning rate scheduler

To handle class imbalance in the training dataset, **class weights** are computed and applied to the loss function (CrossEntropyLoss).

---

### 4.7 Training Process

The model is trained using the MHQA-B dataset. During training:

- Input text is tokenized and converted into tensors
- Forward propagation computes predictions
- Loss is calculated using weighted cross-entropy
- Backpropagation updates model weights
- Learning rate scheduling is applied

Validation is performed after each epoch, and the model with the best validation accuracy is saved.

---

### 4.8 Evaluation Metrics

The model performance is evaluated using standard classification metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Matthews Correlation Coefficient (MCC)

These metrics provide a comprehensive evaluation of classification performance.

---

### 4.9 Model Performance Comparison

Different models were compared based on their performance:

- **BERT** achieved the best performance with:
  - Accuracy: 0.821
  - F1 Score: 0.822
- **ELECTRA** and **DistilBERT** showed competitive performance (~0.80 accuracy)
- **RoBERTa** performed significantly lower compared to other models

Based on these results, **BERT was selected as the final model** for deployment.

---

### 4.10 Final Model Performance

The final BERT model achieved:

- Accuracy: 0.821
- F1 Score: 0.822
- Precision: 0.822
- Recall: 0.822

# 5. IMPLEMENTATION

## 5.1 Tools and Technologies Used

The project is implemented using the following tools and technologies:

- **Programming Language**: Python
- **Frameworks/Libraries**: PyTorch, Hugging Face Transformers, Scikit-learn, Pandas, NumPy
- **Visualization Libraries**: Matplotlib, Seaborn
- **Development Platform**: Google Colab
- **Hardware Support**: GPU (CUDA-enabled environment)

---

## 5.2 Configuration

The model training and experimentation were carried out using the following configuration:

- **Model**: BERT (bert-base-uncased)
- **Maximum Sequence Length**: 256
- **Batch Size**: 16
- **Number of Epochs**: 2
- **Learning Rate**: 2e-5
- **Optimizer**: AdamW
- **Learning Rate Scheduler**: Linear scheduler with warmup
- **Loss Function**: CrossEntropyLoss with class weights
- **Random Seed**: 42

These hyperparameters were selected to balance performance and training efficiency while avoiding overfitting.

---

## 5.3 Development Environment Setup

The implementation was carried out in Google Colab with GPU acceleration enabled. Required libraries were installed and imported prior to execution. The environment ensures efficient training of transformer-based models using CUDA support.

---

## 5.4 Implementation Code

```
!pip install transformers datasets scikit-learn tqdm -q

import os
```

```
import torch

import random

import numpy as np

import pandas as pd

from tqdm import tqdm

from sklearn.metrics import accuracy_score, f1_score, classification_report

from sklearn.preprocessing import LabelEncoder

from sklearn.utils.class_weight import compute_class_weight

from transformers import AutoTokenizer, AutoModelForSequenceClassification,
get_linear_schedule_with_warmup

from torch.utils.data import Dataset, DataLoader

from torch.optim import AdamW

# -------- CONFIG --------

class Config:

    MAX_LEN = 256

    BATCH_SIZE = 16

    EPOCHS = 2

    LR = 2e-5

    SEED = 42

def set_seed(seed=42):

    random.seed(seed)

    np.random.seed(seed)

    torch.manual_seed(seed)

set_seed(Config.SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# -------- LOAD DATA --------

mhqa_df = pd.read_csv("/content/mhqa_gold.csv")

mhqa_b_df = pd.read_csv("/content/mhqa_pseudo.csv")
```

```
TEXT_COLUMN = "question"

LABEL_COLUMN = "topic"

mhqa_df = mhqa_df[[TEXT_COLUMN, LABEL_COLUMN]].dropna()

mhqa_b_df = mhqa_b_df[[TEXT_COLUMN, LABEL_COLUMN]].dropna()

mhqa_df[TEXT_COLUMN] = mhqa_df[TEXT_COLUMN].str.lower().str.strip()

mhqa_b_df[TEXT_COLUMN] = mhqa_b_df[TEXT_COLUMN].str.lower().str.strip()

# -------- LABEL ENCODING --------

label_encoder = LabelEncoder()

all_labels = pd.concat([mhqa_df[LABEL_COLUMN], mhqa_b_df[LABEL_COLUMN]])

label_encoder.fit(all_labels)

mhqa_df[LABEL_COLUMN] = label_encoder.transform(mhqa_df[LABEL_COLUMN])

mhqa_b_df[LABEL_COLUMN] = label_encoder.transform(mhqa_b_df[LABEL_COLUMN])

NUM_LABELS = len(label_encoder.classes_)

# -------- SPLIT --------

train_df = mhqa_b_df.copy()

val_df = mhqa_df.sample(frac=0.5, random_state=42)

test_df = mhqa_df.drop(val_df.index)


# -------- DATASET --------

class MHQADataset(Dataset):

    def __init__(self, texts, labels, tokenizer):

        self.texts = texts.tolist()

        self.labels = labels.tolist()

        self.tokenizer = tokenizer

    def __len__(self):

        return len(self.texts)

    def __getitem__(self, idx):

        enc = self.tokenizer(
```

```
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=Config.MAX_LEN,
            return_tensors='pt'
        )
        return {
            "input_ids": enc["input_ids"].squeeze(0),
            "attention_mask": enc["attention_mask"].squeeze(0),
            "labels": torch.tensor(self.labels[idx], dtype=torch.long)
        }
# -------- MODEL --------
MODEL_NAME = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS
).to(device)
# -------- DATALOADERS --------
train_loader = DataLoader(
    MHQADataset(train_df[TEXT_COLUMN], train_df[LABEL_COLUMN], tokenizer),
    batch_size=Config.BATCH_SIZE,
    shuffle=True
)
val_loader = DataLoader(
    MHQADataset(val_df[TEXT_COLUMN], val_df[LABEL_COLUMN], tokenizer),
    batch_size=Config.BATCH_SIZE
```

```
)
test_loader = DataLoader(
    MHQADataset(test_df[TEXT_COLUMN], test_df[LABEL_COLUMN], tokenizer),
    batch_size=Config.BATCH_SIZE
)
# -------- CLASS WEIGHTS --------
weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_df[LABEL_COLUMN]),
    y=train_df[LABEL_COLUMN]
)
class_weights = torch.tensor(weights, dtype=torch.float).to(device)
loss_fn = torch.nn.CrossEntropyLoss(weight=class_weights)
# -------- OPTIMIZER --------
optimizer = AdamW(model.parameters(), lr=Config.LR)
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=len(train_loader) * Config.EPOCHS
)
# -------- TRAIN --------
best_acc = 0
for epoch in range(Config.EPOCHS):
    model.train()
    total_loss = 0
    for batch in train_loader:
        optimizer.zero_grad()
        ids = batch["input_ids"].to(device)
```

```
        mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)
        outputs = model(input_ids=ids, attention_mask=mask)
        loss = loss_fn(outputs.logits, labels)
        loss.backward()
        optimizer.step()
        scheduler.step()
        total_loss += loss.item()
    # -------- VALIDATION --------
    model.eval()
    preds, labels_list = [], []
    with torch.no_grad():
        for batch in val_loader:
            ids = batch["input_ids"].to(device)
            mask = batch["attention_mask"].to(device)
            outputs = model(input_ids=ids, attention_mask=mask)
            pred = torch.argmax(outputs.logits, dim=1)
            preds.extend(pred.cpu().numpy())
            labels_list.extend(batch["labels"].numpy())
    acc = accuracy_score(labels_list, preds)
    if acc > best_acc:
        best_acc = acc
        torch.save(model.state_dict(), "/content/model.pt")
# -------- TEST --------
model.eval()
```

# 6. RESULTS AND ANALYSIS

## 6.1 Model Performance Comparison

To evaluate the effectiveness of different transformer-based models, multiple architectures were tested, including BERT, ELECTRA, DistilBERT, and RoBERTa. The models were compared using key performance metrics.

**Table 6.1: Model Comparison**

| Model | Accuracy | Precision | Recall | F1 Score | Rank |
|---|---|---|---|---|---|
| **BERT** | 0.821 | 0.820 | 0.820 | 0.822 | 1 |
| ELECTRA | 0.802 | 0.806 | 0.802 | 0.802 | 2 |
| DistilBERT | 0.797 | 0.798 | 0.796 | 0.795 | 3 |

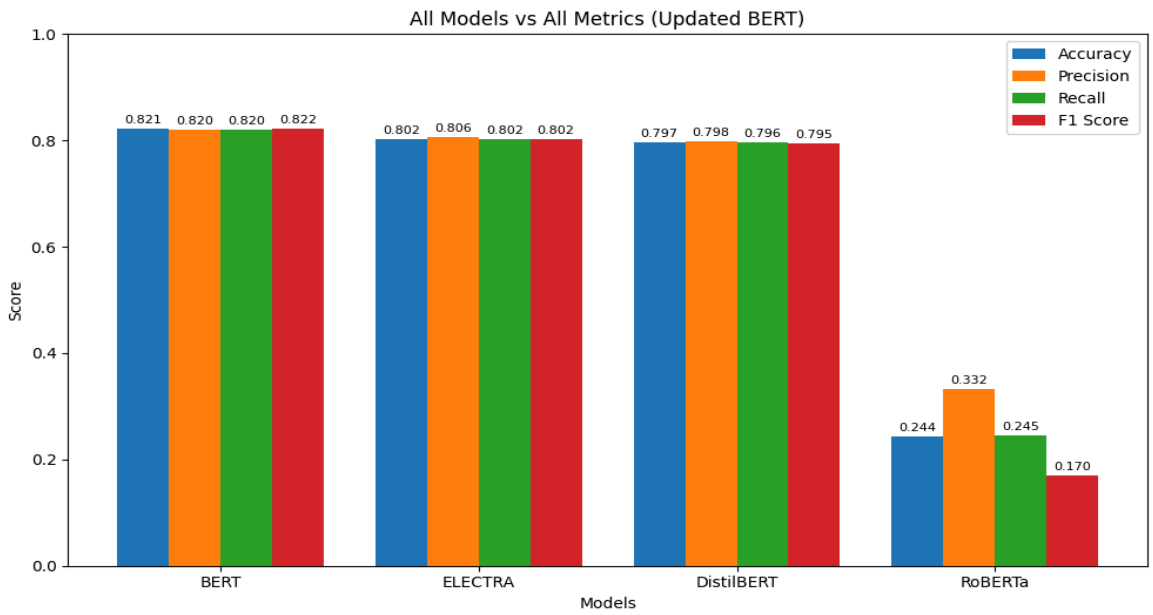
From the comparison, it is evident that BERT achieved the highest performance across all primary metrics, making it the most suitable model for this task.

---

## 6.2 Visualization of Model Performance

The performance of all models across different metrics is visualized using bar charts and heatmaps.

- The bar chart clearly shows that BERT outperforms other models in terms of accuracy and F1-score.
- ELECTRA and DistilBERT demonstrate competitive performance with slightly lower scores.
- RoBERTa performs significantly worse, indicating poor generalization for this dataset.

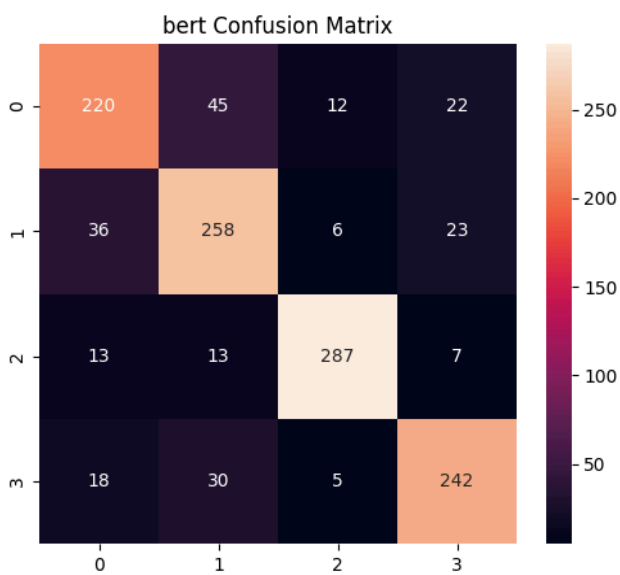



---

### 6.3 Confusion Matrix Analysis

Key detailed insights into classification performance across different classes.

- The model performs exceptionally well for **Obsessive/Compulsive Disorders (Class 2)**.
- Some confusion is observed between **Anxiety (Class 0)** and **Depression (Class 1)** due to overlapping linguistic patterns.
- Minor misclassifications occur in the Trauma category, but overall accuracy remains high.



---

### 6.4 Training Performance

The training process demonstrates consistent improvement across epochs:

**Epoch 1**

- Train Loss: 0.540
- Validation Accuracy: 0.800

**Epoch 2**

- Train Loss: 0.414
- Validation Accuracy: 0.816

The decrease in training loss and increase in validation accuracy indicate effective learning and convergence of the model.

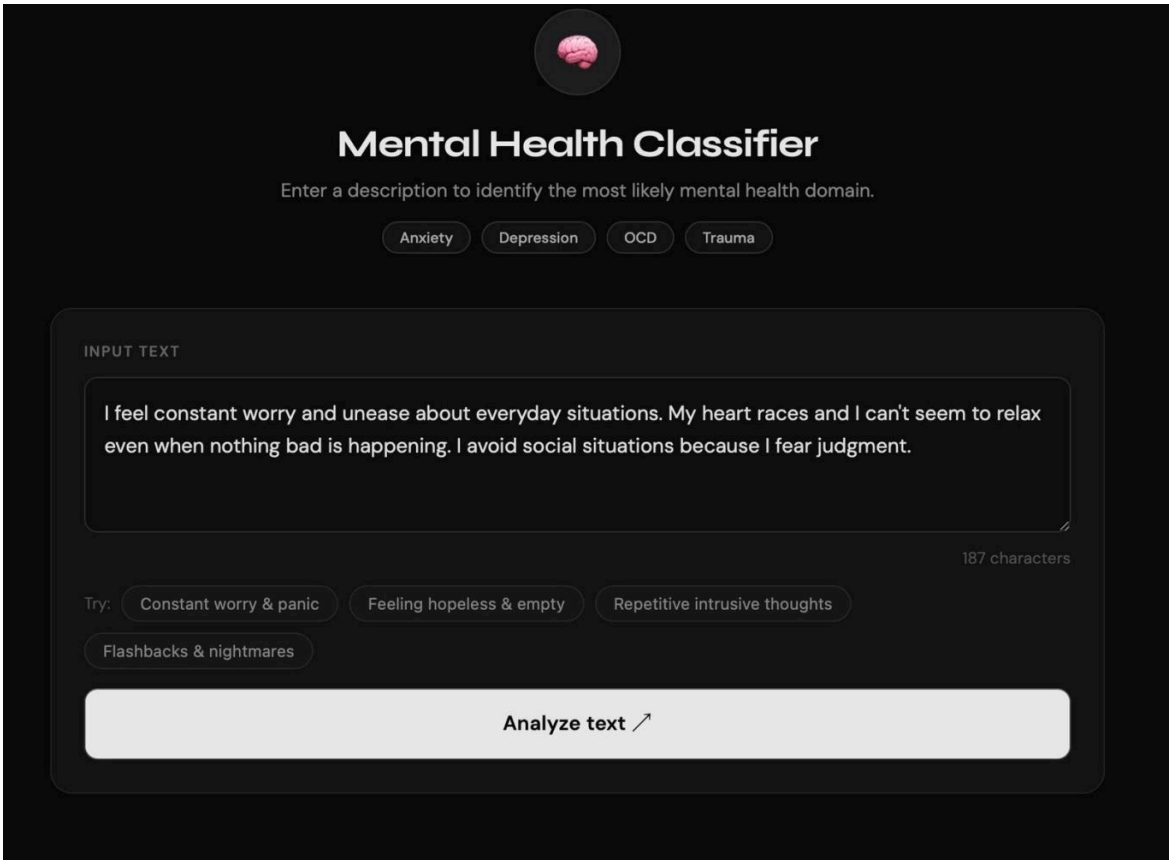
---

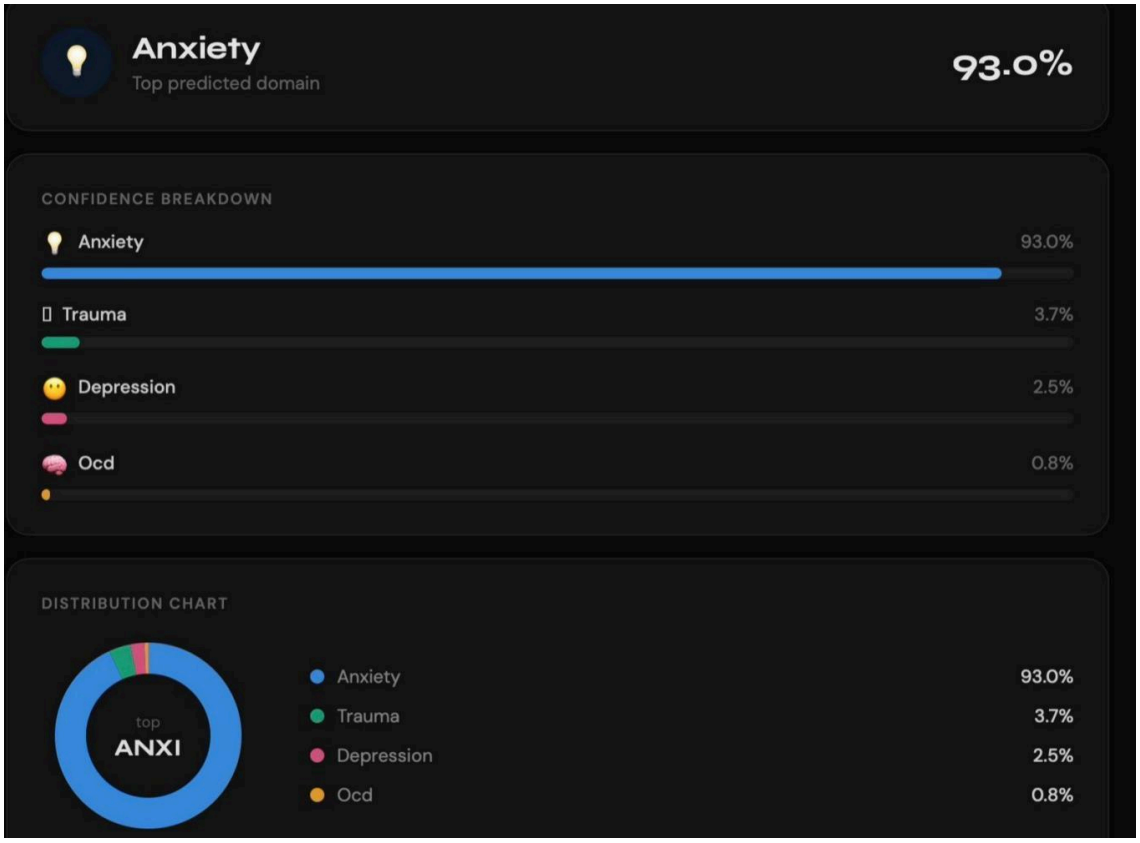
### 6.5 Key Findings

- BERT outperforms other transformer-based models for this task
- The model achieves high accuracy and balanced performance across categories
- Minor confusion exists between semantically similar domains
- Training shows stable convergence with improved validation accuracy
- The system demonstrates strong potential for real-world mental health classification applications

---

### 6.6 UI using Flask


Mental Health Classifier
Enter a description to identify the most likely mental health domain.
Anxiety
Depression
OCD
Trauma
INPUT TEXT
I feel constant worry and unease about everyday situations. My heart races and I can't seem to relax even when nothing bad is happening. I avoid social situations because I fear judgment.
187 characters
Try:
Constant worry & panic
Feeling hopeless & empty
Repetitive intrusive thoughts
Flashbacks & nightmares
Analyze text ↗


Anxiety
Top predicted domain
93.0%
CONFIDENCE BREAKDOWN
Anxiety 93.0%
Trauma 3.7%
Depression 2.5%
Ocd 0.8%
DISTRIBUTION CHART
top
ANXI
Anxiety 93.0%
Trauma 3.7%
Depression 2.5%
Ocd 0.8%

## 7. CONCLUSION

This project successfully developed a transformer-based Natural Language Processing system for classifying mental health-related queries into predefined domains. By leveraging the BharatGen MHQA and MHQA-B datasets, the model was trained on large-scale pseudo-labeled data and evaluated on high-quality human-annotated data, ensuring both scalability and reliability.

Among the models evaluated, BERT (bert-base-uncased) demonstrated the best performance, achieving an accuracy of 0.821 and an F1 score of 0.822. The model effectively captured contextual and semantic information from textual queries, enabling accurate classification across multiple mental health domains.

The results indicate that transformer-based models are highly effective for domain-specific text classification tasks. The system also demonstrated good generalization on unseen data, with only minor confusion between closely related categories such as Anxiety and Depression.

Overall, this project highlights the potential of AI-driven approaches in mental health analysis and provides a strong foundation for building intelligent support systems that can assist in organizing and understanding mental health-related information.

---

## 8. FUTURE WORK

While the current system achieves strong performance, several improvements and extensions can be explored in future work:

- **Incorporating More Advanced Models**: Exploring newer architectures such as RoBERTa-large, DeBERTa, or domain-specific pre-trained models to further improve accuracy.
- **Handling Class Imbalance More Effectively**: Applying advanced techniques such as data augmentation or focal loss to improve performance on underrepresented classes.
- **Multilingual Support**: Extending the model to handle multiple Indian languages to make the system more inclusive and applicable in real-world scenarios.
- **Real-Time Deployment**: Developing a web or mobile-based application to enable real-time classification of mental health queries.
- **Integration with Chatbots**: Incorporating the model into conversational AI systems for automated mental health support and guidance.
- **Explainability and Interpretability**: Adding mechanisms to explain model predictions, which is crucial for sensitive domains like mental health.
- **Use of Real-Time Data**: Incorporating live or streaming data sources to make the system more dynamic and adaptable.

With these enhancements, the system can evolve into a more robust and practical tool for mental health analysis and support.

# 9. APPENDIX

## 9.1 Model Training Notebooks

The complete implementation and training pipelines are provided through the following Google Colab notebooks:

- **All Model Trainings (BERT, ELECTRA, DistilBERT, RoBERTa):** https://colab.research.google.com/drive/1P7LLT8fOwyp5qxGd0fGV1s8MdpIZ7tHY?usp=sharing
- **Proposed Model Training (Final BERT Model):** https://colab.research.google.com/drive/1LuemJ6yHI62BHxCYi-cqp4aMY0_cE4Er?usp=sharing

These notebooks include complete code for data preprocessing, model training, evaluation, and result generation.

---

## 9.2 Reproducibility Note

The provided notebooks can be executed in Google Colab with GPU support enabled. All necessary dependencies are installed within the notebooks, ensuring reproducibility of results.

## 10. REFERENCES


1. BharatGen MHQA Dataset. Available at:
   https://aikosh.indiaai.gov.in/home/datasets/details/bharatgen_mhqa_dataset.html
2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019).
   *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.*
   Proceedings of NAACL-HLT.
3. Hugging Face Transformers Library Documentation. Available at:
   https://huggingface.co/docs/transformers
4. Pedregosa, F., et al. (2011).
   *Scikit-learn: Machine Learning in Python.*
   Journal of Machine Learning Research.
5. Paszke, A., et al. (2019).
   *PyTorch: An Imperative Style, High-Performance Deep Learning Library.*
   Advances in Neural Information Processing Systems.
6. Clark, K., Luong, M. T., Le, Q. V., & Manning, C. D. (2020).
   *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators.*
7. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019).
   *DistilBERT: A distilled version of BERT.*
8. Liu, Y., et al. (2019).
   *RoBERTa: A Robustly Optimized BERT Pretraining Approach.*